

Experimental demonstration of coupled learning in elastic networks

Lauren E. Altman,^{1,*} Menachem Stern¹,[✉] Andrea J. Liu,¹ and Douglas J. Durian^{1,2}

¹*Department of Physics & Astronomy, University of Pennsylvania, Philadelphia, Pennsylvania 19104, USA*

²*Department of Mechanical Engineering and Applied Mechanics, University of Pennsylvania, Philadelphia, Pennsylvania 19104, USA*

 (Received 15 November 2023; revised 19 June 2024; accepted 1 August 2024; published 20 August 2024)

Coupled learning is a contrastive local learning scheme for tuning the properties of individual elements within a network to achieve desired functionality of the system. It takes advantage of physics both to learn using local rules and to “compute” the output response to input data, thus enabling the system to perform decentralized computation without the need for a processor or external memory. We present three proof-of-concept mechanical networks of increasing complexity, and demonstrate how they can learn tasks such as self-symmetrization and node allostery via iterative tuning of individual spring rest lengths. These mechanical networks could feasibly be scaled and automated to solve increasingly complex tasks, hinting at a new class of smart metamaterials.

DOI: [10.1103/PhysRevApplied.22.024053](https://doi.org/10.1103/PhysRevApplied.22.024053)

I. INTRODUCTION

Learning is a physical process by which a system evolves to exhibit a desired behavior. Even algorithms such as artificial neural networks, which are trained via global optimization algorithms such as gradient descent [1], are carried out in digital hardware, incurring a significant energy cost.

Another approach to learning uses computational global optimization (e.g., by backpropagation [2]) to learn or train but “computes,” or calculates outputs in response to inputs (performs inference) in hardware by applying boundary conditions as inputs and using physics to propagate information through the system to “compute” an output response. Such a hybrid approach was used to produce laboratory materials with a desired localized strain response to a localized strain applied elsewhere [3,4], following earlier ideas [5]. Pretrained artificial neural networks have been implemented in physical systems [6] to perform more complex tasks. Most recently, Lee *et al.* [7] used computation to train a network of elastic components with tunable stiffnesses using voice coils. However, hybrid approaches can fail when the model used to compute updates does not approximate the real system well enough. Even for an excellent model, training on the computer becomes more expensive and less accurate as the system increases in size. As a result, there is a substantial advantage to using approaches that avoid computation altogether during training.

Computational global optimization uses global information about the state of the entire network to compute gradients, in order to determine how the system should evolve in the next training step. Biological learning systems, as well as physical learning systems that eschew processors, do not have access to such global information as they evolve, and so must instead rely on decentralized learning schemes based on local update rules to achieve desired behaviors [8]. In such systems, learning is better described as an emergent phenomenon in a complex material. Decentralized local learning has multiple advantages over the computational global optimization approach [9], including power efficiency [10,11], scalability of processing speed with network size [12], and robustness to damage [13,14]. In this approach, training occurs entirely *in situ* on a local, elementwise level.

Specifically, consider physical networks of nodes connected by edges, in which edges update in a distributed manner based on their own responses and properties, so that the time required to compute an update does not significantly increase as the number of edges increases. The poor scaling and computational expense of traditional neural networks that use global rules is one of the major bottlenecks to progress in the field of artificial learning, and raises concerns about energy consumption [15]. Networks that learn “on the fly” via local training schemes also benefit from robustness to damage: if part of the network is lost, the distributed elements can learn a new solution to the desired task [14].

Approaches such as directed aging [16–18], contrastive learning [19,20], and nudged contrastive learning such as equilibrium propagation [21–23] and coupled learning

*Contact author: laltman2@sas.upenn.edu

[14,24,25] can be implemented in experiment alone, relying on each edge's response to an applied boundary condition to tune properties of individual edges. Pashine and coauthors used directed aging to produce laboratory networks with negative Poisson's ratios [16,17], and Arinze and coauthors used local rules akin to directed aging to train thin elastic sheets to fold in desired ways in response to force patterns [26,27]. Directed aging, however, is limited in its use cases because it does not minimize the true cost function of the training task unless the desired response is close to the untrained response.

Approaches based on contrastive learning [19,20] compare a network's internal state in response to different sets of boundary conditions. In particular, nudged contrastive local learning approaches either reduce to gradient descent in the limit of infinitesimal nudges [21] or have large enough projections onto the direction of gradient descent [24] to solve complex tasks effectively [22,23,25,28]. A variety of approaches for implementing these ideas in hardware have been advanced [23,29,30]. Coupled learning has been demonstrated in the lab [14,31,32] to learn various tasks, including classification, but so far only Pashine [20] has used contrastive learning to obtain systems with a desired mechanical behavior (negative Poisson's ratio).

A mechanical network with tunable mechanical edge properties that implements nudged contrastive learning poses several challenges for laboratory construction. The physical degrees of freedom in a mechanical network, namely, the node positions, are vectorial in nature, rather than scalar like the voltages at nodes of an electrical network. This increases the dimensional complexity of the system. A mechanical network is also spatially embedded, so the architecture and connectivity of the network must be carefully chosen for proper network coordination and to avoid collision of neighboring edges as well as mechanical instabilities such as frustration or buckling. It is also a nontrivial engineering problem to develop mechanical elements with adjustable learning degrees of freedom.

A recent work introduces directed springs [33] whose stiffness evolves in response to its physical behavior. Here we introduce a different approach, in which we tune each edge's *rest length*, rather than its *stiffness*. In principle, either set of learning degrees of freedom can be effective in learning; Refs. [16–18] refer to models with tunable stiffness as k models and those with tunable rest lengths as ℓ models. Note that stiffness is directly analogous to conductance in electrical networks, while rest length has no direct analogy and is unique to mechanical systems, so is interesting to study in its own right.

In our approach, a turnbuckle is connected to a Hookean spring at its end. By adjusting the length of the turnbuckle, the effective rest length of the combination spring-turnbuckle edge is altered. Since updates are not automated, but rather made manually in our prototype, it is imperative to choose architectures that minimize the

number of tunable edges while still exhibiting a diversity of tunable behaviors. Here we provide three laboratory demonstrations of experimental mechanical networks that can be trained using coupled learning with adjustable rest length components, and we compare them through simulation. We show that the three architectures effectively span the range of possible states to achieve the desired behavior quickly, smoothly, and effectively. Our experiments demonstrate how real-world considerations like friction and buckling cause deviations from simulated learning behavior, and highlight unexpected difficulties of applying coupled learning to mechanical systems. Despite these limitations, our initial successes show promise for learning more complex functionalities in larger networks, and can serve as a guide for future implementations of elastic learning systems.

II. COUPLED LEARNING FOR SPRING-TURNBUCKLE SYSTEMS

We construct a mechanical network such as that depicted in Fig. 1(a). Each edge of this network is an extension spring with the same stiffness k and rest length l connected to a rigid turnbuckle. The turnbuckle for spring i allows it to have an adjustable length L_i , as drawn schematically in the inset of Fig. 1(a). The total node-node separation of the edge, including the length of the rigid part, is s_i , such that the spring's extension past its rest length is $s_i - (l + L_i)$. The mechanical energy associated with this edge is given by Hooke's law as

$$u_i = \frac{1}{2}k[s_i - (l + L_i)]^2. \quad (1)$$

The total mechanical energy in the network is the sum of the elastic energies of all edges,

$$U = \sum_j u_j, \quad (2)$$

and serves as the “physical cost function” that the system automatically minimizes with respect to the node positions, namely, the “physical degrees of freedom,” subject to the imposed boundary conditions.

We wish to train this network to achieve some desired behavior in the positions of its nodes. This is what is referred to as a “motion task.” We select certain nodes to be “inputs” and “outputs” for our task, depicted in Fig. 1(a) as blue and red filled circles, respectively. When the input node is at position $\vec{x}^I$, we want the output node's position $\vec{x}^O$ to be at some desired value, $\vec{x}^D$.

We train for this behavior by iteratively adjusting the turnbuckle lengths at each edge according to the coupled learning algorithm [24]. Here, the node positions, or physical degrees of freedom, equilibrate so that there is no net force on each edge, while the rest lengths act as “learning

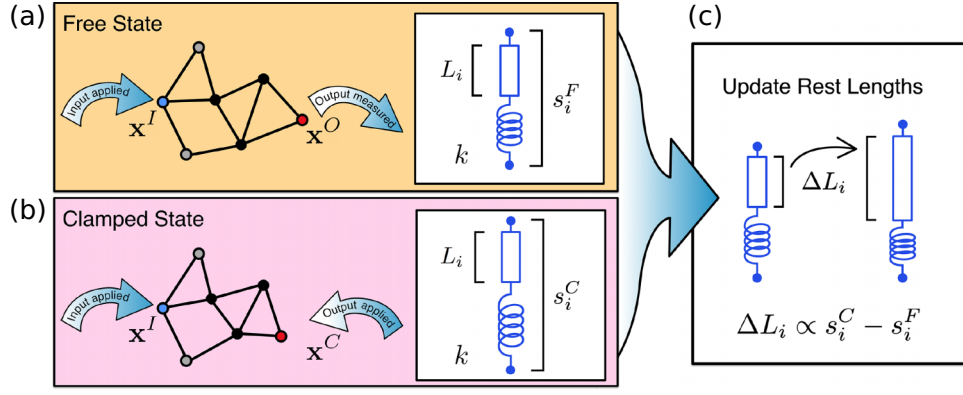


FIG. 1. Schematic detailing the coupled learning algorithm for an arbitrary elastic network. A mechanical network is constructed such that each i th edge consists of a spring with stiffness k connected to a turnbuckle with adjustable rest length L_i . The springs act as the edges of the network, and their connection points are nodes. Specific nodes are chosen as “inputs” and “outputs” for the network to learn a desired task. Some nodes (gray) are fixed to prevent translation and rotation of the entire network, and the remaining nodes move in response to imposed boundary conditions. (a) In the free state, the position of the input node $\vec{x}^I$ is enforced, and the position of the output node $\vec{x}^O$ is measured. Each i th edge has length (i.e., node-node separation) s_i^F . (b) In the clamped state, the input node’s position is still fixed at $\vec{x}^I$ and the output node is “clamped” to position $\vec{x}^C$. Each i th edge has length s_i^C . (c) By locally comparing the lengths of each edge, the update rule in Eq. (6) determines how L_i evolves.

degrees of freedom” to be adjusted during training. In the free state, Fig. 1(a), we apply the input boundary condition to the network by fixing the position of the input node, $\vec{x}^I$, and allowing all other physical degrees of freedom to equilibrate. The output node’s position $\vec{x}^O$ is measured, along with the extensions of each edge in the network, s_i^F . In the clamped state, Fig. 1(b), we once again apply the input boundary condition, but now we also enforce a fixed value for the position of the output node by “nudging” it towards the desired position $\vec{x}^D$. The clamped output value $\vec{x}^C$ generally takes the form

$$\vec{x}^C = \vec{x}^F + \eta(\vec{x}^D - \vec{x}^F), \quad (3)$$

where the “nudge factor” $0 < \eta \leq 1$ is a hyperparameter of the training scheme. In all experiments below, we take a full nudge of $\eta = 1$, so the output node is clamped at the desired position at each learning step. The lengths s_i^C of each edge in this state are also recorded.

In coupled learning, the system evolves by comparing the mechanical energy of the network in the free and clamped states. Learning is achieved when the energy in the clamped state U^C is equal to the energy in the free state U^F . In the absence of nonlinear mechanical effects like buckling, the difference between these energies, which we refer to as the “learning contrast function,” is always non-negative because the clamped state is more strongly constrained than the free state, $U^C - U^F \geq 0$. Analogous to the machine-learning approach of minimizing a loss function like the mean squared error, the rest lengths evolve by descending along the gradient of the contrast

function:

$$\frac{dL_i}{dt} \propto -\frac{\partial}{\partial L_i} \left[\sum u_j^C - u_j^F \right] \quad (4)$$

$$\begin{aligned} &= -\frac{\partial}{\partial L_i} (u_i^C - u_i^F) \\ &= k(s_i^C - s_i^F). \end{aligned} \quad (5)$$

Since the learning contrast function was not squared, the partial derivative in Eq. (4) picks out only the $j = i$ term, which is readily simplified to Eq. (5) using Eq. (1). Thus, we arrive at the following general discrete learning rule for each edge’s update over a learning step for any network of identical spring-turnbuckle edges:

$$\Delta L_i = \alpha(s_i^C - s_i^F). \quad (6)$$

Here α is a per-step learning rate that we set in the experiment, and $(s_i^C - s_i^F)$ is the difference in the clamped and free lengths of the edge being updated. This simple rule is purely local: each edge updates according only to its behavior, irrespective of how other edges change upon clamping. In the experiments presented below, we train with different learning rate parameters in the range $0.1 < \alpha \leq 1$. Iterative updates should drive the global learning contrast function to zero in order to achieve the desired motion function. Appendix D details the conditions under which the local and global update rules align.

While our Eq. (6) update rule is spatially local, it is not temporally local because the learning rule requires simultaneous information about the system in two states. The experimental implementation in an electrical resistor

network [14] was able to circumvent this issue by building identical twin networks to run the free and clamped states simultaneously. By contrast, the mechanical network is embedded in space, posing difficulties for constructing twin two-dimensional networks side by side, and an impossibility entirely for three-dimensional implementations. Therefore, our approach must rely on temporal memory of the spring extensions between the free and clamped states.

III. MOTION DIVIDER

Analogous to the voltage divider in the electrical network case, our first demonstration is a “motion divider” network, or two tunable-rest length springs connected in series. A photograph of the network is shown in Fig. 2(a). We construct the network so that it is hanging vertically under gravity from a fixed point, thereby restricting the motion to be along the axis of gravity. We define the one-dimensional position $y = 0$ to be at the fixed hanging point, and $y > 0$ measures the position downwards from this origin. The first spring is connected to the $y = 0$ fixed

point via a turnbuckle of length L_1 , terminating at position y_1 . The second turnbuckle of length L_2 then connects to the second spring, which terminates at position y_2 . We choose y_2 to be the input node of our system and y_1 to be the output node. Both springs have equal stiffnesses k and natural rest lengths l , so that the Eq. (6) learning rule applies. The forces acting on springs in series are equal, so $k(y_1 - l - L_1) = k(y_2 - y_1 - l - L_2)$ holds and can be solved for the output node position as

$$y_1 = \frac{1}{2}y_2 + a, \quad (7)$$

where $a = \frac{1}{2}(L_1 - L_2)$. Choosing some desired fixed value for a defines a trainable task for our system. This is equivalent to a machine-learning linear regression problem with one variable coefficient [34]. Note that there is no unique solution for L_1 and L_2 ; only their difference is trained for. Like typical machine-learning algorithms, our system is overparameterized and thus has multiple solutions.

A. Apparatus

Each unit cell of our network consists of an extension spring coupled to a turnbuckle in series, as shown diagrammatically in Fig. 1 and photographically in Fig. 2(a). The spring has a stiffness of $k = 13$ N/cm and a rest length of $l = 5.5$ cm (Grainger 5108N536), while the turnbuckle has a range of lengths between $L^{\min} = 12$ cm and $L^{\max} = 16$ cm (eocvt M4 Stainless Steel 304). Updates are made on the system via turns of the turnbuckle, where each half-turn results in a change in length of $\Delta L/\text{turn} = 0.079$ cm. The total effective rest length of the unit cell object is therefore $l + L_i$. These unit cells are connected together using keyrings, which serve functionally as the nodes of our network. The system is clamped at the nodes manually by inserting a small rod through the keyrings and fixing its position up or down using a clamp on a vertical pole. Measurements were made manually by the experimenter using a ruler.

B. Results

Figure 2(b) shows the evolution of the node couplings over the course of training. The network is trained multiple times to achieve different values for the goal state a . The turnbuckle lengths were not reset after each training; instead, the network was able to relearn a new task without initialization. The evolution of the learning degrees of freedom, L_1, L_2 , is shown in Fig. 2(c). Each time we change the training task a , we also choose a different learning rate to demonstrate that the network may be trained at different time scales.

The training time required to reach the desired state is determined by both the learning rate and the distance in parameter space between the network’s initial and final states. The physical and learning degrees of freedom

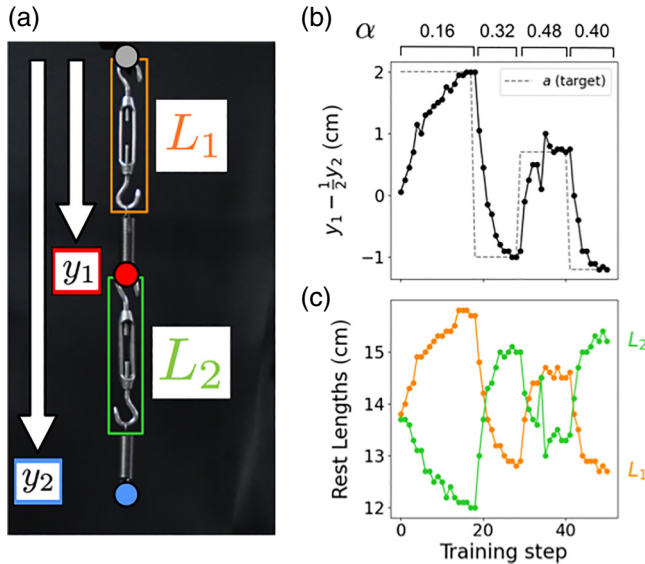
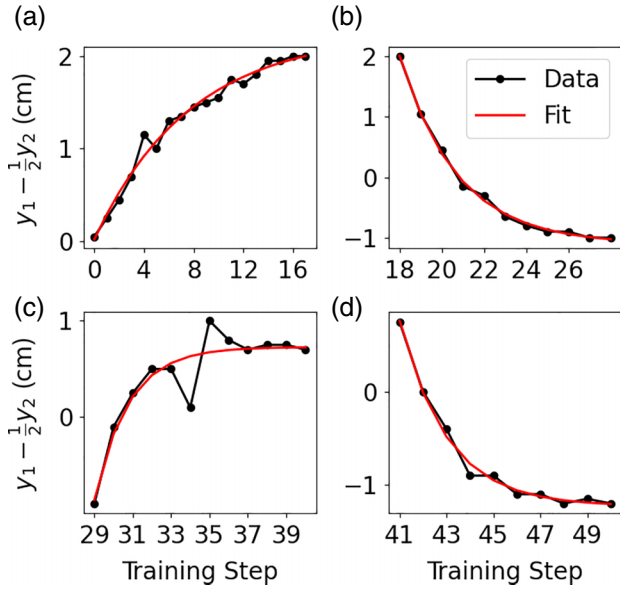


FIG. 2. Training a mechanical motion divider to obtain desired behavior in the form of an additive constant a . (a) Photograph of the experimental apparatus. Two springs in series are hung vertically from a fixed point. The positions of the two nodes relative to the fixed point, y_1 and y_2 , serve as the target and source for the training task, respectively. The learning degrees of freedom, L_1, L_2 , determine the relationship between the node positions. (b) Results of training the network four consecutive times. Different goal values for a (dashed gray line) and learning rates α were used for each consecutive training. (c) Evolution of the learning degrees of freedom over the course of training. As the network approaches the goal state, L_1 (orange) evolves inversely to L_2 (green). Rest lengths were not reset to their initial values after each successful training.



Trial	True a (cm)	Fit a (cm)	True α	Fit α
(a)	2.00	2.25	0.16	0.13
(b)	-1.00	-1.11	0.32	0.37
(c)	0.70	0.73	0.48	0.56
(d)	-1.20	-1.23	0.40	0.49

FIG. 3. Comparison of motion divider learning dynamics with the theoretical prediction. The time evolution of the physical behavior for each of the four trials from Fig. 2(b) is separately fit to an exponential form and values for the goal state a and the learning rate α are obtained. The table (bottom) compares these fitted values with the true experimental values for a and α for each of the four trials.

are expected to evolve exponentially and asymptotically approach their desired values over the course of training. For a full derivation of the learning dynamics, see Appendix A. Figure 3 presents each of the four trials of training separately. Each of these is fit to the derived time evolution (A12), where νt has been replaced by αn with n the number of training steps. The fits are overlaid on the data in red. Deviations of the data from their expected behavior, such as is the case in Fig. 3(c), are likely due to human operator error. The Fig. 3 table compares the true values for a and α with those obtained from the exponential fit, and finds good agreement, with an average error of 12%. By comparing fitted values for a with the true values, we may also obtain an estimate of the measurement error, where $\epsilon \sim \langle \Delta a \rangle = 0.1$ cm.

IV. SYMMETRY NETWORK

We further demonstrate our system's ability to learn more complex tasks by constructing a two-dimensional network within a frame. The network is shown schematically and photographically in Figs. 4(a) and 4(b). Four

of the edges are fixed to a rigid 45×40 cm² frame constructed from 80/20[®], with the fifth central edge connecting the two internal nodes of the system. Edges are typically stretched past their rest length value by about 6 cm to prevent buckling. Edges are labeled by a number and nodes are denoted as the source (blue) and target (red).

The goal is to train this network to become left-right symmetric, which occurs when $L_1 = L_2$ and $L_4 = L_5$. This task may be encoded in the physical degrees of freedom (node positions) of the network with the condition

$$x_{\text{target}} = x_{\text{source}} = 0 \quad \text{for any } y_{\text{source}} \text{ value,}$$

where $x = 0$ is defined to be the midline between the fixed nodes on either side. This choice of task is not unique: other tasks can be “solved” by the desired symmetric state; see Appendix C. We may then train the network for this one-input, two-output task using a single data point.

A. Experiment

The symmetry network was constructed using the same unit cell design as in the motion divider in Sec. III. The springs used here have a natural rest length of $l = 5.5$ cm and a stiffness of 27.8 N/cm (Grainger 1NAA2). The external frame of the network was constructed from 80/20 parts and eyeholes screwed into the rail serve as the fixed nodes.

Training this network requires only one input-output pair. Both outputs, x_{source} and x_{target} , have desired values that are directly central in the frame. Since training can occur for any choice of the input value, y_{source} , we choose the equilibrium position of this node in its initial state for simplicity.

It is important to note that the physical degrees of freedom of a single node are mixed: the “source” node acts as an input in the y direction and as an output in the x direction. The coupled learning algorithm allows for this seemingly strange coupling, and the network is still trainable under this scheme. The mixing of degrees of freedom was achieved experimentally by aligning the nodes along 80/20 tracks, as can be seen in the photos in Figs. 4(a) and 4(b). In the free state, we fix the input value for the training task, y_{source} , by attaching the node to a slide-in nylon tool hanger that slots into a horizontal 80/20 track, so its vertical position is fixed. The node may still move along the horizontal direction with a small coefficient of static friction $\mu \approx 0.1$ for nylon on dry aluminum. The target node is left unconstrained, so both its degrees of freedom can freely equilibrate. In the clamped state, we fix both degrees of freedom of the bottom node, $\{y_{\text{source}}, x_{\text{source}}\}$, by placing stoppers along the horizontal track described previously. The horizontal position of the top node, x_{target} , is restricted by placing it on a vertical track using the same tool hanger as above, while its vertical position y_{target} is allowed to equilibrate freely.

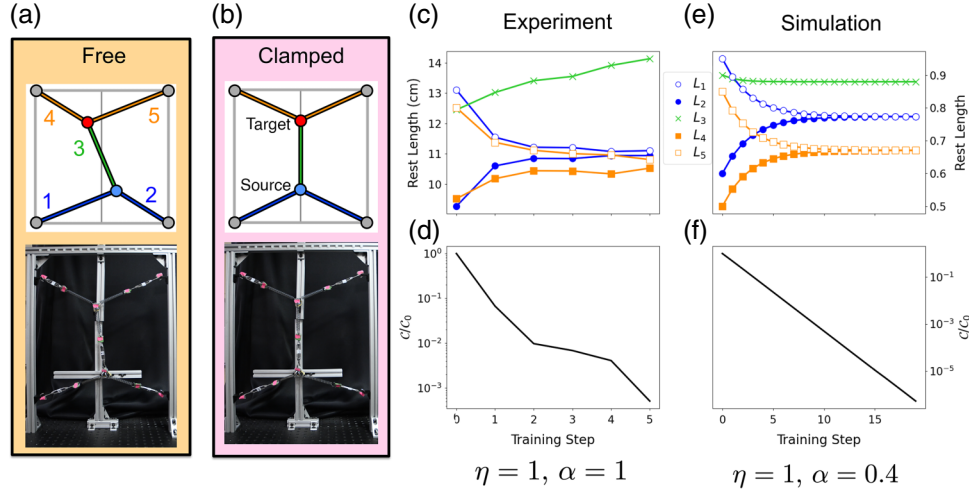


FIG. 4. Training a two-dimensional network for symmetry. (a),(b) Schematics and photographs of the free and clamped states of the network for a given initial configuration. Edges are labeled by a number [(a), top]. Photographs of the experimental apparatus show the marker tracking system and clamping mechanism. (c),(d) Experimental training results. Rest lengths (c) converge to the symmetric state within experimental precision. The cost $\mathcal{C} = U_C - U_F$ (d) decreases by 3 orders of magnitude. (e),(f) Simulation training results. The rest lengths (e) evolve such that $L_1 = L_2$ and $L_4 = L_5$. Cost (f) decreases exponentially by 5 orders of magnitude.

Since this network requires measurements in two dimensions, we automate the measurement process using a digital camera. As can be seen in the photographs in Figs. 4(a) and 4(b), we attach pink markers to all six nodes, as well as to the five points of connection between the turnbuckles and springs. The camera captures a three-channel color image of the entire network, where we choose a beneficial white balance (2500 K) and tint (M6.0) to maximize the contrast between the markers and the remainder of the image. We then split the color channels and subtract the green channel from the red, leaving us with a one-channel image with high-intensity values at the pixels associated with markers and low intensity everywhere else. The image is then binarized using a threshold intensity value of 100, which is in between the well-separated high- and low-intensity values. The pixel values that have been identified after binarization are then clustered into 11 clusters using a k -means algorithm [35]. The centroid of each cluster is determined to be the (x, y) position of the associated marker. This allows us to track all node positions, spring extensions, and turnbuckle lengths at every stage of the experiment. For a given photograph with 11 markers such as those in Fig. 4, the full processing and tracking takes approximately 0.7 s on a standard Macbook Pro.

The results of training the physical network are shown in Figs. 4(c) and 4(d). The learning degrees of freedom evolve such that L_1 comes to meet L_2 and L_4 meets L_5 within a measurement error of 0.28 cm in only five training steps. The time evolution of these rest lengths follows a similar trend as in the motion divider, where the largest adjustments happen in the early training steps and updates

become asymptotically small as the network approaches the learned state. See Appendix A for more information about learning dynamics. The success of training may also be measured by the cost function, $\mathcal{C} = U_C - U_F$. Figure 4(d) shows that the cost decreases exponentially from its initial value by about 3 orders of magnitude over the course of training.

The middle edge L_3 does not exhibit exponential time-evolution behavior, but rather drifts upward over the course of training by a total of 1.5 cm, or 37.5% of the range of values. This linear evolution suggests some systematic experimental bias within the system, which may be a result of friction or mechanical instability. Since L_3 has no bearing on the desired state, the network can learn the task for any value of L_3 .

B. Simulation

There are many different ways to train for symmetry in our two-dimensional network. Our goal for this task is a specified internal state of the network, but the coupled learning scheme requires that we specify a *behavior* in the nodes that is satisfied by this internal state, of which there are multiple options. We therefore use simulation support to explore the different iterations of our task and examine their behavioral dynamics. We may also use simulation to modify the aspect ratio of the frame, as the angles of the edges at the internal nodes contribute to the coupling strength of the learning signal. Having done this, we chose the training task and geometry that allowed for the most efficient evolution to the learned state for our experimental demonstration.

We simulated training on this network using the FIRE optimization algorithm to determine the network's node positions when boundary conditions are applied [36]. Here, each edge was allowed to vary between 0.5 and 1.0 length units, and all edge stiffnesses were fixed at the same value, 1. Initial edge lengths were selected at random with validation that the network was sufficiently detuned from its goal state.

The behavior that was used to train the network is a one-input, two-output task, but we could have alternatively defined a one-input, one-output task that is equally satisfied by the symmetric state of the network. Training on the one-input, one-output task was successful in simulation, but required very long training times that were unfeasible for an experiment with manual operation. In this case, L_4 and L_5 evolve quickly to their desired values due to their strong coupling to the target node. However, L_1 and L_2 are only connected directly to the source node, which does not move in position very much between the free and clamped states, and therefore evolve very slowly. For

more information, see Appendix C. The two-output task allows for a strong learning signal for all of the edges in the network.

Training was performed using a full nudge $\eta = 1$ and Eq. (6) with a learning rate of $\alpha = 0.4$. Successful training is possible with a higher learning rate, but we have artificially slowed the training to better examine the learning dynamics. Figure 4(e) shows the evolution of the edges' rest lengths during the course of training. The outer edges evolve smoothly to meet their desired state, $L_1 = L_2$ and $L_4 = L_5$, in approximately ten training steps.

The middle edge's rest length L_3 has no bearing on the desired goal state, yet it still evolves during training, particularly in the first few training steps. The desired state is not dependent on L_3 , but the cost function is, as this is the edge that directly connects the two relevant nodes. While evolution of L_3 is not required for the network to learn, allowing it to vary during training according to the coupled learning update rule helps the network descend down the cost landscape more quickly.

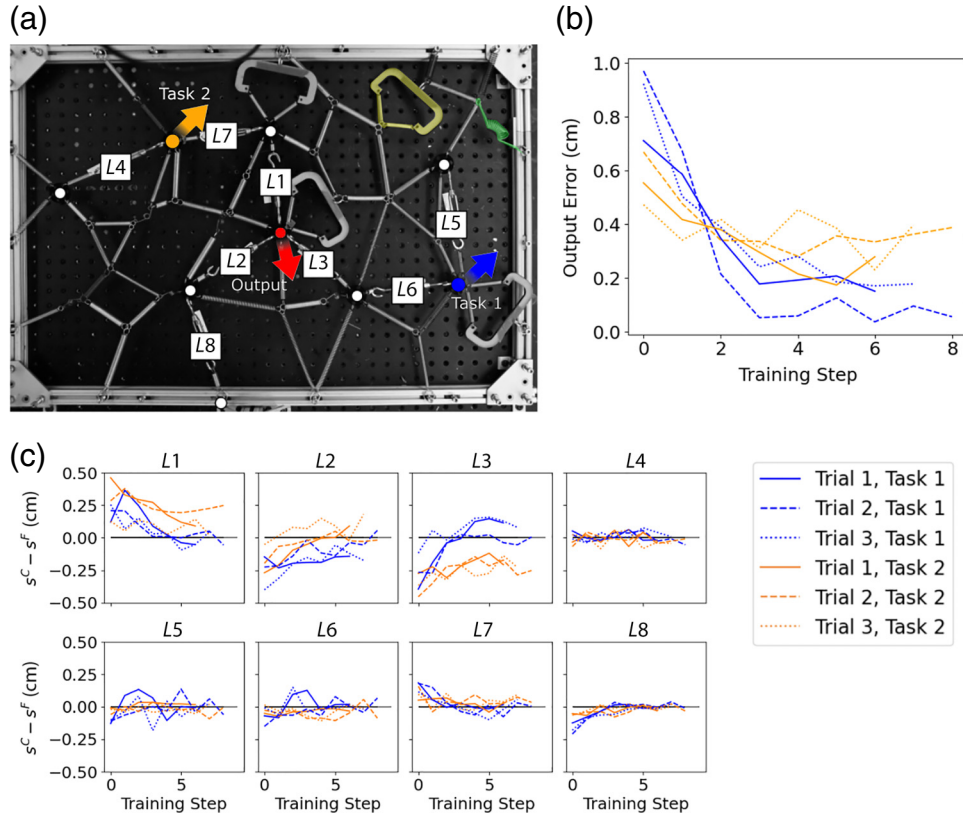


FIG. 5. Training of a nonlinear elastic network for node allosteric tasks. (a) Schematic of the network. There are 39 nodes and 59 edges, eight of which have tunable rest length, labeled L_1 – L_8 . Nonlinear mechanical elements are included, such as torsion springs, highlighted in green, and springs whose force-displacement relationship is cubic, highlighted in yellow. The input nodes for tasks 1 and 2 are labeled in blue and orange, and the shared output node is labeled in red. (b) Error in the position of the output node over training. The network is trained for both allosteric tasks simultaneously in three separate trials. In each trial, the network is initialized to a different configuration. Error in task 1 (blue) is reduced to a greater degree than that in task 2 (orange), indicating that the network can more readily learn this task. (c) Learning rule updates over training for each tunable edge. Edges with larger initial updates are more important for learning the task, and can be correlated with proximity to the output node.

In the experimental trial, Fig. 4(c), L_3 systematically drifts upward, rather than the asymptotic decrease that is seen in Fig. 4(e). While this likely inhibited the speed of training, the network was still able to converge on the desired configuration of rest lengths. This lends credence to the coupled learning algorithm's robustness even in nonideal experimental conditions. In our contrastive local scheme, training can fail to converge for individual edges on a local level, but the network as a whole can succeed in learning the desired behavior.

V. RANDOMIZED NONLINEAR NETWORK

As a final example that would be highly nontrivial to model, we construct a third and much larger random network, composed of a mix of adjustable spring-tumbuckle edges and nonadjustable linear as well as nonlinear edges. It consists of 59 edges and 39 nodes, depicted in Fig. 5(a). This network is stretched inside of a frame with 13 nodes fixed to the boundary. The network is arranged in the plane perpendicular to the direction of gravity, but its internal tension is strong enough to render effects from gravity irrelevant to the experimental procedure. Only eight of these edges are tunable, labeled with an “L” in the schematic. The majority of springs (tunable or not) in this network are linear, with a variety of native rest lengths and stiffnesses (Grainger 3HPU5). This network also includes nonlinear springs, including torsion springs (Grainger 3HPR8, 3HPD1), highlighted in green in Fig. 5(a), and compound springs whose force-displacement relationship is cubic, highlighted in yellow. The inclusion of nonlinear edges expands the class of potential behaviors a network can learn, and serves as an analogue to a reservoir computer, in the sense that there are many nonadjustable unknown elements influencing the physical behavior of our system [37].

We train this network simultaneously for two different node allosteric tasks. The input nodes for each of the two tasks are labeled in blue and orange in Fig. 5(a), respectively. Both tasks share the same output node, labeled in red. Training data for each task are a single pair of two-dimensional input and output node positions, and the nodes are clamped with $\eta = 1$ using the screw holes in the optical table. For task 1, the input node is stretched away from the output node to a position of (77.6, 45.9) cm from the upper left corner of the frame, and for task 2, the input is pushed towards the output to a position of (26.6, 23.4) cm. For both tasks, the output node was clamped to a position of (44.5, 38.5) cm. Because our edges consist of extension springs that go slack under compression, task 1 is inherently easier to train for than task 2, as applying the input adds tension to the network, rather than releasing it.

During training, we alternate between the two tasks at each learning step. Updates are made using the learning rule given in Eq. (6) with $\alpha = 0.84$. Training is halted

when the error for each task falls below a measurement threshold, or when rest length updates average to zero over two learning steps. We perform training in three separate trials, each time initializing the network's tunable edge lengths to different random configurations.

The results of these trials are shown in Fig. 5(b). In each case, the mean output error between both tasks is reduced by a factor of about 5 over the course of training, which takes between 10 and 20 training steps. The network is able to learn task 1 more quickly and accurately than task 2, finding lower-error solutions in fewer training steps. Among the three trials, training for task 1 achieved an average final output error of 0.13 cm, while training for task 2 only reached 0.35 cm.

Figure 5(c) depicts the evolution of the learning updates for each tunable edge, separated by tasks. Here we can see each edge's role in learning the tasks. Edges that make larger initial updates, such as edges $L1$ and $L3$, have a stronger influence on the output node position, and would therefore change more significantly when the clamped

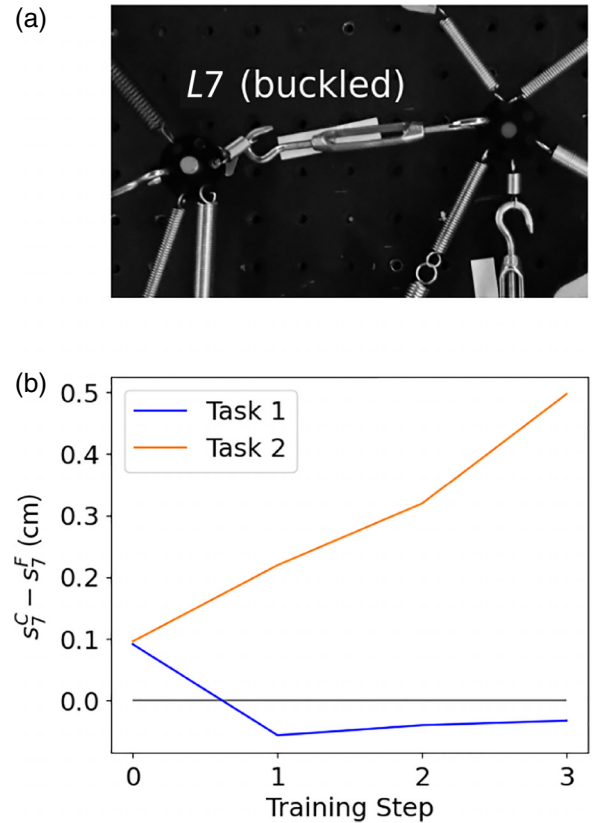


FIG. 6. Instance of a failed training as a result of mechanical instability. (a) Photograph of tunable edge $L7$ in a buckled state as a result of the free-state boundary conditions in task 2. This occurs if $L7$ is too long. (b) Learning rule updates for edge $L7$ for each task. During training for task 2, the update rule steadily increases the length of $L7$, further decoupling it from the rest of the network.

boundary conditions are applied. An edge's significance loosely correlates with its proximity to the output node. We can also see how the network "favors" learning task 1 over task 2. While each task is trainable to zero error individually, training for both simultaneously can result in updates with respect to each task that cancel each other out, resulting in no net change in the learning degree of freedom, as is the case with $L3$ in trial 1.

In performing these trials, we note that training success is somewhat sensitive to the initial configuration of the network. Figure 6 depicts a case where training of task 2 failed as a result of mechanical instability in the network. If edge $L7$ is initialized to be too long then it will buckle when the free and clamped boundary conditions of task 2 are applied. When an edge is buckled, it no longer imparts force on the network, and therefore has no effect on the position of the output node. The learning rule will still update this edge, even though these updates are no longer correlated with the energy at that edge. This results in updates to $L7$ continuously increasing, rather than converging to zero. This behavior was only observed when training task 2.

VI. CONCLUSION

We have demonstrated that laboratory realizations of mechanical networks can be trained for desired behaviors using the contrastive coupled learning algorithm. Through three different spring network architectures, we have shown that learning is achievable despite clear differences in the learning dynamics between simulation and experiment, or when a simulation of the physical system would be infeasible. While functionality is limited due to their small size, these initial experiments are sufficient to demonstrate the promise of contrastive local learning in larger-scale physical materials.

The choice of a spring-turnbuckle implementation has limitations, however, and also highlights the pitfalls of our implementation. These edges require manual tuning, which limits the size of the network due to the labor required by the supervisor. Furthermore, the requirement of a supervisor undercuts the local, decentralized nature of the learning scheme, and lessens the benefits associated with locality, such as scalability and compute time. Another key limitation to our approach is that each of our elastic edges is only capable of tensile forces. As a result, under certain boundary conditions on the network, some edges can undergo mechanical instabilities such as buckling when compressed, effectively decoupling from the network, but still updating under the contrastive learning rule. Each of our experiments were therefore performed under tension, with most edges stretched noticeably beyond their rest length.

The choice of rest length as the learning degree of freedom also poses limitations, especially when coupled with

an apparatus that can only impart force under tension. The presence of a nonzero rest length introduces nonlinearity in the force-displacement relation, a feature that is not present in resistive or flow networks. However, this nonlinearity is most prevalent when the spring is near its equilibrium length. Our extension springs learn more effectively when far above their equilibrium length, so we cannot take full advantage of this nonlinearity, or of the potential for more complex tasks enabled by nonlinearity, with this setup. See Appendix B for more information.

Future implementations of mechanical contrastive local learning networks will include automated components that could perform local sensing and actuation to self-adjust. Furthermore, a mechanical element that learns by updating its stiffness rather than its rest length would allow for force as well as motion tasks. A promising example of such an element is an electronically controlled tunable-stiffness spring [38]. These springs are constructed of predominantly cheap or three-dimensional printed parts, operate by feeding concentric coils into or out of an elastic ring to modify the stiffness, and use flex and force sensors to locally sense their elastic energies. The modularity and ease of construction of these tunable springs should allow for significant scalability to more complex architectures and tasks.

Despite the engineering challenges, we see potential for local contrastive learning in elastic systems. The inherent nonlinearity present as a result of a nonzero rest length as well as mechanical instabilities pose opportunities to train for complex nonlinear tasks. Mechanical networks are capable of behaviors and material properties for which there is no analog in electrical contrastive local learning networks, such as multistability and auxeticity [4,39]. The proof of principle we have presented here suggests that our decentralized approach could allow for learning networks to scale up. "Smart" mechanical network materials that can learn mechanical tasks and properties locally and in real time would have far-reaching applications from soft robotics to material science.

ACKNOWLEDGMENTS

We thank Samuel Dillavou, Ben Pisanty, Maggie Miller, Cynthia Sung, and Shivangi Misra for their helpful discussions and contributions. This work was supported by NSF grants MRSEC/DMR-1720530 (L.E.A., D.J.D.) and MRSEC/DMR-2309043 (L.E.A., D.J.D.), the U.S. Department of Energy, Office of Basic Energy Sciences, Division of Materials Sciences and Engineering Award DE-SC0020963 (M.S.), and the Simons Foundation (# 327939 to A.J.L.). D.J.D. and A.J.L. thank CCB at the Flatiron Institute, a division of the Simons Foundation, as well as the Isaac Newton Institute for Mathematical Sciences under the program "New Statistical Physics in Living Matter" (EPSRC Grant EP/R014601/1) for support

and hospitality while a portion of this research was carried out. This work was initiated and performed in part at Aspen Center for Physics, which is supported by National Science Foundation Grant PHY-2210452.

APPENDIX A: MOTION DIVIDER LEARNING DYNAMICS

For the motion divider, we can explicitly write out and solve for the learning dynamics. In this task, we choose an input value y_2 for the position of the end node. The system is stretched and y_2 is held fixed for both the free and clamped states during the entire course of training. The position of the middle node, y_1 , serves as the output. It will change over the course of training, both between free and clamped states and between training steps as L_1 and L_2 evolve. In the free state, force balance gives the position of the middle node as

$$y_1^F = \frac{1}{2}y_2 + \frac{1}{2}(L_1 - L_2). \quad (\text{A1})$$

Before training, this differs from the desired output value given by

$$y_1^D = \frac{1}{2}y_2 + a \quad (\text{A2})$$

for whatever value the user has chosen for a . During training, the goal is to adjust the turnbuckle lengths so that $y_1^F \rightarrow y_1^D$, which happens when $(L_1 - L_2)/2 \rightarrow a$. The update rule to achieve this is based on comparing the free state to the clamped state, where the output node is clamped by nudge factor $0 < \eta \leq 1$ toward the desired position

$$y_1^C = \eta y_1^D + (1 - \eta)y_1^F. \quad (\text{A3})$$

The general discrete update rule, Eq. (6), may now be evaluated by computing the node-node separations of each edge in free and clamped states using the above expressions. This gives

$$\Delta L_1 = \alpha \eta \left(a - \frac{L_1 - L_2}{2} \right), \quad (\text{A4})$$

$$\Delta L_2 = -\Delta L_1. \quad (\text{A5})$$

Note that these updates are equal and opposite, and vanish when learning is achieved. Also, note that the nudge factor η and update factor α equivalently affect the rate of learning. Thus, in experiments, we take $\eta = 1$ and vary α without loss of generality. It should be emphasized that in the lab we use only Eq. (6) to make the updates based on measurement of node-node separations in free and clamped states. In the lab, physics “computes” y_1^F automatically, whereas here in the appendix we additionally bring force balance to bear in order to predict learning dynamics.

To obtain differential equations from the predicted discrete update rules, one might guess that the time derivatives of L_1 and L_2 should be proportional to the right-hand sides of Eqs. (A4) and (A5). This turns out to be true. To see this, recall that the elastic energy in the clamped state must be greater than that in the free state and the difference serves as the coupled learning contrast function. For the motion divider, it can be explicitly computed using the above positions and simplifies to

$$\mathcal{C} = U_C - U_F = k\eta^2 \left(a - \frac{L_1 - L_2}{2} \right)^2. \quad (\text{A6})$$

Not only is this intrinsically positive, it is proportional to the loss function $\mathcal{L} = (y_1^D - y_1^F)^2$. See Appendix D for more details about when this proportionality is expected. Therefore, gradient descent on $\mathcal{C}$ is perfectly aligned with gradient descent on $\mathcal{L}$. The resulting coupled learning dynamical equations, $dL_i/dt = -\gamma d\mathcal{C}/dL_i$, simplify to

$$\frac{\partial L_1}{\partial t} = \nu \left(a - \frac{L_1 - L_2}{2} \right), \quad (\text{A7})$$

$$\frac{\partial L_2}{\partial t} = -\frac{\partial L_1}{\partial t}, \quad (\text{A8})$$

where $\nu = \gamma k\eta^2$ is a rate constant with units of 1/time. These are in agreement with intuition from the discrete version above. The time-evolution equations, Eqs. (A7) and (A8), may be directly integrated to obtain

$$L_1(t) = e^{-\nu t} \left(\frac{L_1^0}{2} - \frac{L_2^0}{2} - a \right) + a + \frac{L_1^0}{2} + \frac{L_2^0}{2}, \quad (\text{A9})$$

$$L_2(t) = e^{-\nu t} \left(a - \frac{L_1^0}{2} + \frac{L_2^0}{2} \right) - a + \frac{L_1^0}{2} + \frac{L_2^0}{2}, \quad (\text{A10})$$

where L_1^0 and L_2^0 are the initial rest lengths of the two edges, respectively. The network stops evolving when

$$\left| \frac{L_1 - L_2}{2} - a \right| \leq \epsilon, \quad (\text{A11})$$

where ϵ is the desired training accuracy. Note that the analytic solutions imply that the training error

$$\frac{L_1(t) - L_2(t)}{2} - a = e^{-\nu t} \left[\frac{L_1^0 - L_2^0}{2} - a \right] \quad (\text{A12})$$

decreases exponentially in time. Thus, the required duration is set by the training rate ν , as well as the initial conditions and the desired accuracy.

APPENDIX B: RELATIVE AND ABSOLUTE NODE POSITIONS

In each demonstration presented in this work, we train using input and output node positions in an external reference frame, rather than relative to their own equilibrium positions. This is a necessary choice for successful training in an elastic system using the rest length as the learning degree of freedom. One way to see this is to note that training in coupled learning generally only requires reference to the network under two sets of boundary conditions, the free state and the clamped state, while this training protocol would require reference to a third state, the equilibrium state. This is only the case because we are using the rest length as the learning degree of freedom, since the equilibrium state positions change with each update step. When training on spring stiffnesses, the equilibrium state remains constant over training, so relative position training is allowed in this case.

We use the simple case of a motion divider for an explicit demonstration of this constraint. In Sec. III and Appendix A, we defined this system's equation of state as $y_1 = \frac{1}{2}y_2 + \frac{1}{2}(L_1 - L_2)$ and training data were defined with an input position y_2 and desired output position $y_1^D = \frac{1}{2}y_2 + \alpha$, with α as a chosen fixed value. Both input and output positions are relative to a fixed origin, which we define as the boundary node position, depicted as the gray node in Fig. 2(a). We could have instead defined each node's equilibrium (unstretched) positions,

$$y_1^0 = L_1, \quad (\text{B1})$$

$$y_2^0 = L_1 + L_2, \quad (\text{B2})$$

and used the motions relative to these equilibrium positions as our training data,

$$y_2 = y_2^0 + A, \quad (\text{B3})$$

$$y_1^D = y_1^0 + B, \quad (\text{B4})$$

with A and B as chosen fixed values. In the free state, the target node is in position

$$y_1^F = \frac{1}{2}(y_2^0 + A + L_1 - L_2) = L_1 + \frac{A}{2}, \quad (\text{B5})$$

and in the clamped state,

$$\begin{aligned} y_1^C &= \eta y_1^D + (1 - \eta)y_1^F \\ &= \eta(L_1 + B) + (1 - \eta)\left(L_1 + \frac{A}{2}\right) \\ &= L_1 + \frac{A}{2} + \eta\left(B - \frac{A}{2}\right). \end{aligned} \quad (\text{B6})$$

Evaluating the learning rule for each edge according to Eq. (6), we get

$$\Delta L_1 = \alpha \eta \left(B - \frac{A}{2}\right), \quad (\text{B7})$$

$$\Delta L_2 = \alpha \eta \left(\frac{A}{2} - B\right). \quad (\text{B8})$$

The learning rule update for each edge is now a constant value, and will therefore never converge regardless of the values of L_1 and L_2 . The training error,

$$y_1^D - y_1^F = B - \frac{A}{2}, \quad (\text{B9})$$

is also a fixed value and does not depend on the learning degrees of freedom.

Another key difference to note is that changing an edge's rest length does significantly change the force delivered across that edge, $F = k(x - l)$, when the edge's extension is far from the equilibrium point, $x \gg l$. Altering an edge's stiffness affects the proportionality of force to motion, but the rest length only affects the position of zero force. Therefore, the greatest potential for change in the network's behavior by tuning the rest length is to alter which edges are under extension or compression. As we have previously stated, our turnbuckle-based apparatus is constructed such that edges buckle when $x < l$, so our experiments are typically performed such that each edge is under extension and far from its equilibrium point. Section V and Fig. 6 demonstrate how training can fail when edges buckle. In the simulation of larger networks, training for relative motions using the rest length can sometimes succeed, but because the motions do not change significantly, the network cannot be detuned very far from the solution. Differences in node positions between the free and clamped states are therefore well below the measurement and clamping precision available to us in experiments.

APPENDIX C: ONE-OUTPUT SYMMETRY TRAINING

As expressed in Sec. IV, there are multiple ways to define a task in the physical degrees of freedom that is satisfied by the desired state, $L_1 = L_2$ and $L_4 = L_5$. Such a well-defined task of this nature could be

$$x_{\text{target}} = 0 \quad \text{for any two } y_{\text{source}} \text{ values.}$$

Unlike the two-output training, which only required a single data point to train, this training scheme requires that we choose two values of y_{source} , which we alternate between during training steps.

We train the network using this training scheme in the simulation, using the equilibrium node position $y_{\text{source}}^{\text{eq}}$ and

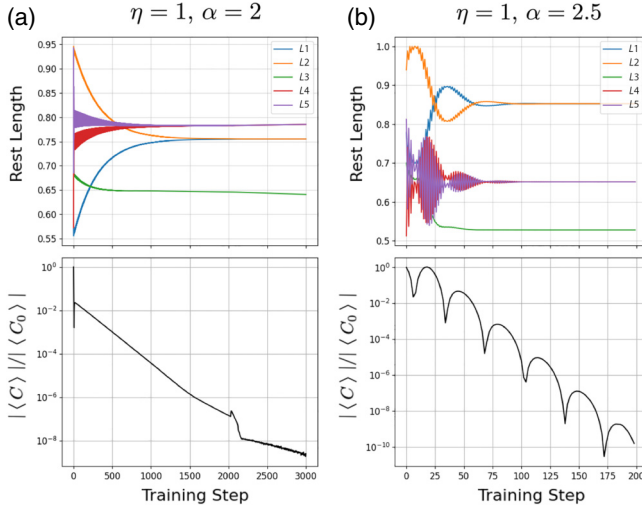


FIG. 7. Simulated training of the one-output symmetry training with varying hyperparameters. Learning degrees of freedom oscillate over the course of training, and even the cost function does not decrease monotonically. Training with $\eta = 1$ and $\alpha = 2$ (a) takes approximately 2000 steps to reach the minimal cost value. Training with a high learning rate $\alpha = 2.5$ (b) reduces the training time to only 200, but displays unstable learning dynamics.

the bottom edge of the frame as the two input data points. The output data points are both at $x = 0$, in the center of the frame. Choosing two input data points that are well separated increases the speed of learning because the nodes move a greater distance between the alternating training steps.

Two simulated trainings of this one-output task are shown in Fig. 7. Both trainings use the full nudge, $\eta = 1$, and the learning rate α is varied. Figure 7(a) depicts the standard training regime, where $\alpha = 2$. Training takes an excessively long 2000 steps to reach the minimum cost, with training times increasing with decreasing learning rate. The rest lengths L_4 and L_5 exhibit oscillatory dynamics over the course of training, while L_1 and L_2 incrementally move towards their goal values. Increasing the learning rate to $\alpha s = 2.5$ moves us into the “overtraining” regime, Fig. 7(b), in which each learning degree of freedom can evolve *past* its goal value in a single training step. Here, the training time is reduced significantly to only 200 steps, but the cost function does not decrease monotonically over the course of training. The cost can increase by up to an order of magnitude in only a few training steps, but on average decreases by 10 orders of magnitude. The rest lengths also exhibit strong oscillatory behavior, and can even switch their symmetry states between training steps. While this type of overtraining is in principle possible to perform experimentally, the unstable learning dynamics make for a nonideal demonstration of coupled learning’s capabilities.

APPENDIX D: ALIGNMENT OF COUPLED LEARNING WITH GRADIENT DESCENT FOR ELASTIC NETWORKS

In this work we showed how coupled learning [24] can be used to train elastic networks for desired responses in experimental conditions. Here we discuss the coupled learning update of Eq. (5) for elastic networks in more detail and discuss theoretical aspects of its experimental implementation.

Consider the node positions of the free state of the network $\mathbf{x}^F$, obtained by physical relaxation of the springs subject to inputs. By definition, the free state is a local energy minimum, so that the energy of a state in its close proximity $\mathbf{x}^F + \delta\mathbf{x}$ is

$$\begin{aligned} U(\mathbf{x}^F + \delta\mathbf{x}; L_i) &\approx U(\mathbf{x}^F; L_i) + \frac{1}{2}\delta\mathbf{x}^T H(\mathbf{x}^F; L_i)\delta\mathbf{x} \\ &= U^F + \frac{1}{2}\delta\mathbf{x}^T H^F \delta\mathbf{x}, \end{aligned} \quad (\text{D1})$$

where $H(\mathbf{x}^F; L_i) \equiv H^F = d^2U/\partial\mathbf{x}\partial\mathbf{x}$ is the *physical Hessian* of the network, i.e., the matrix of second derivatives of the energy with respect to node positions, evaluated at the free state.

In coupled learning, we define the clamped state with Eq. (3), by clamping the output nodes closer to their desired positions. The physics now minimizes the network energy associated with these clamping constraints, which can be encoded using Lagrange multipliers

$$U = U^F + \frac{1}{2}\delta\mathbf{x}^T H^F \delta\mathbf{x} + \lambda_O[\vec{x}^C - \vec{x}^F - \eta(\vec{x}^D - \vec{x}^F)]_O. \quad (\text{D2})$$

The subscripts O indicate that the Lagrange multipliers only associate with the physical degrees of freedom constrained in the output. Assuming a small nudge ($\eta \ll 1$), the clamped state minimizing Eq. (D2) is very close to the free state and belongs to the same energy basin. In this regime, one can solve for the clamped state given the free-state positions $\mathbf{x}^F$ and the clamping constraints.

To do that, we can recast Eq. (D2) by defining an extended “position” vector $\mathbf{y} \equiv [\mathbf{x}, \lambda_O]^T$, and a clamping “force” vector $\mathbf{f}^O \equiv [\mathbf{0}, x_O^D - x_O^F]$. Note that, when no clamping force is applied, $\lambda_O = 0$ and the free state is given by $\mathbf{y}^F = [\mathbf{x}^F, 0_O]^T$. We further construct an output matrix S_O with N (physical degrees of freedom) columns and a number of rows equal to the number of output degrees of freedom N_O . Each row in this matrix has a single nonvanishing component of value 1, at the index associated with the corresponding output physical degree of freedom. With this, we can define a *clamped physical Hessian*

$$H^C = \begin{pmatrix} H^F & S_O^T \\ S_O & 0 \end{pmatrix}. \quad (\text{D3})$$

Using these definitions, we can write the physical optimization problem of Eq. (D2) and its solution (see Ref. [40]) as

$$U = U^F + \frac{1}{2} \delta \mathbf{y}^T H^C \delta \mathbf{y} - \eta \delta \mathbf{y}^T \mathbf{f}^O, \quad (\text{D4a})$$

$$\mathbf{y}^C = \mathbf{y}^F + \eta (H^C)^{-1} \mathbf{f}^O, \quad (\text{D4b})$$

$$U^C = U^F - \frac{1}{2} \eta^2 (x^D - x^F)_O^T (H^C)_{OO}^{-1} (x^D - x^F)_O, \quad (\text{D4c})$$

with $(H^C)^{-1}$ the inverse of the clamped physical Hessian and $(H^C)_{OO}^{-1}$ the negative definite bottom right block of this inverse matrix.

Using the contrastive learning framework, the coupled learning rule is given by the difference between the clamped and free energies, differentiated with respect to the learning degrees of freedom (rest lengths):

$$\begin{aligned} \Delta L_i(\text{CL}) &\sim \eta^{-1} \frac{\partial (U^F - U^C)}{\partial L_i} \\ &\approx (x^D - x^F)_O^T (H^C)_{OO}^{-1} \frac{d}{dL_i} x_O^F. \end{aligned} \quad (\text{D5})$$

A more standard path to training the elastic networks is to choose the learning rule according to gradient descent, i.e., by computing the gradient of the learning cost function $C = \frac{1}{2} (x^D - x^F)_O^2$ with respect to the learning degrees of freedom. It is easy to verify that the gradient descent modification is given by

$$\Delta L_i(\text{GD}) \sim -\frac{dC}{dL_i} = -(x^D - x^F)_O^T \frac{d}{dL_i} x_O^F. \quad (\text{D6})$$

One can see the similarity between the local learning rule afforded by coupled learning and the global gradient computation. The difference between them stems from the insertion of the negative definite matrix $(H^C)_{OO}^{-1}$. In the special case where there is just a single output degree of freedom, this matrix is just a negative scalar, and the coupled learning rule is proportional to that obtained by gradient descent, as discussed for such a case in Appendix A. However, this is no longer the case with more complicated tasks with multiple output degrees of freedom. In such cases the two rules of Eqs. (D5) and (D6) are no longer proportional, as matrix $(H^C)_{OO}^{-1}$ is in general not proportional to the identity matrix. The modification vector would then be misaligned, and in extreme cases one might even have a negative projection on the other (in which case coupled learning would actually locally *increase* the error). However, we numerically found that in high likelihood, the two learning rules are correlated and point in similar directions in the space of learning degrees of freedom [24]. Even if the alignment between them is locally poor, a few steps of the learning process in the wrong direction produce a brief “unlearning spike,” during which the error grows, but then

the two rules align again and effective learning resumes. We have never numerically observed coupled learning fail by being consistently misaligned with the cost gradient, though we cannot rule out the existence of such cases.

Another important qualification to the above considerations is that they are only strictly valid in the linear regime around the free-state configuration. The contrastive learning (and coupled learning theory) assumes that the clamped state is infinitesimally close to the free state, i.e., that small nudges ($\eta \ll 1$) are applied in the clamping. In that limit one can treat the difference between the free and clamped energies as a derivative, and a connection can be drawn between the contrast and the cost function one wishes to minimize. In this $\eta \ll 1$ limit, the free and clamped states are obviously very similar, as the difference between them scale with η [Eq. (D4)]. In systems with thermal noise or uncertain measurements, one cannot use arbitrarily small nudges, as any noise in the measurement would be much larger than the contrast between the two states. In our experiments we thus use large nudges $\eta \approx 1$, which introduces various theoretical complications. For one, the theoretical relationships drawn between the local learning rule and the cost gradient are only valid to first order in η so that large nudges worsen the approximation of the cost gradient afforded by our local learning rule. Even worse, as elastic networks are nonlinear, clamping with large nudges can move the configuration into a different basin in the energy landscape (compared to the free state). When the clamped state can no longer be treated as a perturbation around the free state, the formal justification for contrastive learning is lost, and we can no longer guarantee learning convergence. Such cases often also lead to the aforementioned “unlearning spikes” and growing error. In practice, though, we find that learning in our system is successful in numerical and experimental settings even in these conditions; the network eventually discovers a configuration of the learning degrees of freedom for which the free and clamped states occur in the same energy basin, and coupled learning is successful from there.

-
- [1] Y. LeCun, Y. Bengio, and G. Hinton, Deep learning, *Nature* **521**, 436 (2015).
 - [2] R. Rojas, in *Neural Networks: A Systematic Introduction* (Springer Berlin Heidelberg, Berlin, Heidelberg, 1996), p. 149.
 - [3] J. W. Rocks, N. Pashine, I. Bischofberger, C. P. Goodrich, A. J. Liu, and S. R. Nagel, Designing allostery-inspired response in mechanical networks, *Proc. Natl. Acad. Sci.* **114**, 2520 (2017).
 - [4] D. R. Reid, N. Pashine, J. M. Wozniak, H. M. Jaeger, A. J. Liu, S. R. Nagel, and J. J. de Pablo, Auxetic metamaterials from disordered networks, *Proc. Natl. Acad. Sci.* **115**, E1384 (2018).
 - [5] C. P. Goodrich, A. J. Liu, and S. R. Nagel, The principle of independent bond-level response: Tuning by pruning to

- exploit disorder for global behavior, *Phys. Rev. Lett.* **114**, 225501 (2015).
- [6] L. G. Wright, T. Onodera, M. M. Stein, T. Wang, D. T. Schachter, Z. Hu, and P. L. McMahon, Deep physical neural networks trained with backpropagation, *Nature* **601**, 549 (2022).
- [7] R. H. Lee, E. A. Mulder, and J. B. Hopkins, Mechanical neural networks: Architected materials that learn behaviors, *Sci. Robot.* **7**, eabq7278 (2022).
- [8] M. Stern and A. Murugan, Learning without neurons in physical systems, *Annu. Rev. Condens. Matter Phys.* **14**, 417 (2023).
- [9] S. Denève, A. Alemi, and R. Bourdoukan, The brain as an efficient and robust adaptive learner, *Neuron* **94**, 969 (2017).
- [10] B. Sengupta and M. B. Stemmler, Power consumption during neuronal computation, *Proc. IEEE* **102**, 738 (2014).
- [11] M. Stern, S. Dillavou, D. Jayaraman, D. J. Durian, and A. J. Liu, Training self-learning circuits for power-efficient solutions, *APL Mach. Learn.* **2**, 016114 (2024).
- [12] R. A. John, N. Tiwari, M. I. B. Patdillah, M. R. Kulkarni, N. Tiwari, J. Basu, S. K. Bose, Ankit, C. J. Yu, A. Nirmal, *et al.*, Self healable neuromorphic memtransistor elements for decentralized sensory signal processing in robotics, *Nat. Commun.* **11**, 4030 (2020).
- [13] R. A. McGovern, A. N. V. Moosa, L. Jehi, R. Busch, L. Ferguson, A. Gupta, J. Gonzalez-Martinez, E. Wyllie, I. Najm, and W. E. Bingaman, Hemispherectomy in adults and adolescents: Seizure and functional outcomes in 47 patients, *Epilepsia* **60**, 2416 (2019).
- [14] S. Dillavou, M. Stern, A. J. Liu, and D. J. Durian, Demonstration of decentralized physics-driven learning, *Phys. Rev. Appl.* **18**, 014040 (2022).
- [15] A. de Vries, The growing energy footprint of artificial intelligence, *Joule* **7**, 2191 (2023).
- [16] N. Pashine, D. Hexner, A. J. Liu, and S. R. Nagel, Directed aging, memory, and nature's greed, *Sci. Adv.* **5**, eaax4215 (2019).
- [17] D. Hexner, N. Pashine, A. J. Liu, and S. R. Nagel, Effect of directed aging on nonlinear elasticity and memory formation in a material, *Phys. Rev. Res.* **2**, 043231 (2020).
- [18] D. Hexner, A. J. Liu, and S. R. Nagel, Periodic training of creeping solids, *Proc. Natl. Acad. Sci.* **117**, 31690 (2020).
- [19] J. R. Movellan, in *Connectionist Models*, edited by D. S. Touretzky, J. L. Elman, T. J. Sejnowski, and G. E. Hinton (Morgan Kaufmann, San Mateo, California, 1991), p. 10.
- [20] N. Pashine, Local rules for fabricating allosteric networks, *Phys. Rev. Mater.* **5**, 065607 (2021).
- [21] B. Scellier and Y. Bengio, Equilibrium propagation: Bridging the gap between energy-based models and backpropagation, *Front. Comput. Neurosci.* **11**, 24 (2017).
- [22] J. Kendall, R. Pantone, K. Manickavasagam, Y. Bengio, and B. Scellier, Training end-to-end analog neural networks with equilibrium propagation, *ArXiv:2006.01981*.
- [23] E. Martin, M. Ernoult, J. Laydevant, S. Li, D. Querlioz, T. Petrisor, and J. Grollier, EqSpike: Spike-driven equilibrium propagation for neuromorphic implementations, *iScience* **24**, 102222 (2021).
- [24] M. Stern, D. Hexner, J. W. Rocks, and A. J. Liu, Supervised learning in physical networks: from machine learning to learning machines, *Phys. Rev. X* **11**, 021045 (2021).
- [25] S. Dillavou, B. D. Beyer, M. Stern, A. J. Liu, M. Z. Miskin, and D. J. Durian, Machine learning without a processor: Emergent learning in a nonlinear analog network, *PNAS* **121**, e2319718121 (2024).
- [26] C. Arinze, M. Stern, S. R. Nagel, and A. Murugan, Learning to self-fold at a bifurcation, *Phys. Rev. E* **107**, 025001 (2023).
- [27] M. Stern, C. Arinze, L. Perez, S. E. Palmer, and A. Murugan, Supervised learning through physical changes in a mechanical system, *Proc. Natl. Acad. Sci.* **117**, 14843 (2020).
- [28] B. Scellier, M. Ernoult, J. Kendall, and S. Kumar, in *ICML Workshop on Localized Learning (LLW)* (ICML, Honolulu, Hawaii, 2023).
- [29] V. R. Anisetti, B. Scellier, and J. M. Schwarz, Learning by non-interfering feedback chemical signaling in physical networks, *Phys. Rev. Res.* **5**, 023024 (2023).
- [30] M. J. Falk, J. Wu, A. Matthews, V. Sachdeva, N. Pashine, M. L. Gardel, S. R. Nagel, and A. Murugan, Learning to learn by using nonequilibrium training protocols for adaptable materials, *Proc. Natl. Acad. Sci.* **120**, e2219558120 (2023).
- [31] J. F. Wycoff, S. Dillavou, M. Stern, A. J. Liu, and D. J. Durian, Desynchronous learning in a physics-driven learning network, *J. Chem. Phys.* **156**, 144903 (2022).
- [32] M. Stern, S. Dillavou, M. Z. Miskin, D. J. Durian, and A. J. Liu, Physical learning beyond the quasistatic limit, *Phys. Rev. Res.* **4**, L022037 (2022).
- [33] V. P. Patil, I. Ho, and M. Prakash, Self-learning mechanical circuits, *ArXiv:2304.08711*.
- [34] T. M. Hope, in *Machine Learning*, edited by A. Mechelli and S. Vieira (Academic Press, London, United Kingdom, 2020), p. 67.
- [35] S. Lloyd, Least squares quantization in PCM, *IEEE Trans. Inf. Theory* **28**, 129 (1982).
- [36] E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, and P. Gumbsch, Structural relaxation made simple, *Phys. Rev. Lett.* **97**, 170201 (2006).
- [37] B. Schrauwen, D. Verstraeten, and J. Van Campenhout, in *Proceedings of the 15th European Symposium on Artificial Neural Networks (ESANN)*, Bruges, Belgium, 2007), p. 471.
- [38] S. Misra, M. Mitchell, R. Chen, and C. Sung, in *IEEE International Conference on Robotics and Automation (ICRA)* (IEEE, London, United Kingdom, 2023).
- [39] M. Stern, M. B. Pinson, and A. Murugan, Continual learning of multiple memories in mechanical networks, *Phys. Rev. X* **10**, 031044 (2020).
- [40] M. Stern, A. J. Liu, and V. Balasubramanian, Physical effects of learning, *Phys. Rev. E* **109**, 024311 (2024).